

dMIR Peephole Rewrites for Consensus-Critical EVM JITs with First-Class Carry Operators

Anonymous Author(s)

Anonymous Institution
anonymous@example.com

Abstract. Blockchain JIT peephole rewrites must preserve transaction semantics bit-for-bit; any divergence between consensus-path VMs can cause chain forks. In LLVM IR, extracting carry and overflow from aggregate intrinsic returns fragments carry-dependent rewrites across SSA nodes, blocking single Alive2 queries and forcing implementers to drop such rules, ship them unverified, or hand-prove them one by one. dMIR (DTVM’s middle IR) represents carry and overflow as first-class bit-vector operators, so each dMIR rewrite can be expressed as a single Z3 equivalence query within a two-tier system. An offline Z3 admission and coverage pipeline checks the 70 production dMIR rewrites under fixed-width bit-vector semantics, including carry-chain rewrites enabled by first-class ADC/SBB operators. A separate x86 CgIR layer adds 13 cleanup rewrites that are validated by matched differential testing rather than SMT. The full 83-rule system achieves a 7.24% geometric-mean speedup on 27 evmone-bench workloads, with 5884 differential Cancun tests reporting byte-identical final state and matching gas accounting across the suite (correctness bounded by suite coverage). This design addresses a verification bottleneck for peephole rewrites in consensus-critical engines; more broadly, domain-specific IR with first-class bit-vector operators provides a reusable path for SMT-checkable rule design in such engines.

Keywords: Blockchain information systems · EVM JIT · dMIR · Peephole optimization · SMT equivalence checking · Carry chain · Translation validation

1 Introduction

The execution layer of a blockchain information system is a replicated consensus state machine: every node on the consensus path must produce bit-identical results for the same transaction. The EVM is a widely deployed smart-contract execution environment; when a client enables a JIT, the compiler becomes part of the consensus-critical path. Peephole rewrites in the JIT act directly at machine-code generation, so their correctness bears directly on the determinism of on-chain execution; a wrong equivalence judgment, deployed on only some clients, may trigger a chain fork.

Existing formal verification of EVM peephole rewrites mostly targets the bytecode block-level abstraction. The machine-level patterns emitted by a JIT during code generation—especially the multi-word carry chains that arise when the 256-bit word length is lowered onto 64-bit hardware—sit at a different abstraction layer from bytecode rules and are awkward to validate as single stock LLVM/Alive2 peephole queries. The root cause is how IRs model bit-level side-products. In LLVM IR, basic arithmetic is defined only on N -bit integers; the carry bit must be retrieved through an intrinsic that returns a multi-return tuple of {result, carry}. Rules depending on this side-product can be expressed only through indirect structures in Alive [1] and similar tools; the bottleneck is the IR’s expressiveness, not the SMT solver [2]. For a consensus-critical JIT, an unchecked rule class leaves the engineer with three choices—drop it (performance), ship it without verification (safety), or hand-prove it (scalability).

Our approach: extend the IR with first-class bit-vector operators so that SMT-checkable peephole rules cover a larger surface, without modifying the solver or relaxing the differential-test threshold. In dMIR, the bit-level patterns that previously required indirect multi-return encoding become first-class operators, so the corresponding rules fall within Z3 bit-vector equivalence. The SMT scope is dMIR arithmetic/flag semantics; gas, memory, storage, and x86 equivalence remain outside it.

Our main contributions are as follows. First, we extend dMIR with first-class carry and overflow operators; the four carry-chain rewrites of Table 1 reduce to single Z3 bit-vector queries, replacing the indirect LLVM/Alive2 multi-return tuple encoding. The two-tier rule set contains **70** SMT-checked dMIR rules and **13** differentially tested x86 cleanup rules. Second, an enumerable, Z3-admitted search space covers the production set: a capacity run admits **853** candidates—**98.7%** of algebraic and **21.8%** of carry templates—and a coverage run produces **1,966** admitted rewrites whose canonical left-hand sides match all **70** hand-designed production rules. Third, the full 83-rule set delivers a **+7.24%** geometric-mean speedup on the 27 evmone-bench workloads (95% CI [+2.59%, +12.11%], peak **+25.79%**) and passes **5,884** differential Cancun tests with byte-identical final state and matching gas accounting, with correctness bounded by suite coverage.

The remainder of the paper surveys related work, presents the dMIR bit-vector model and admission pipeline together with the U256 carry-chain motivation, reports end-to-end performance and correctness, and concludes.

2 Related Work

2.1 Peephole Optimization Frameworks

We position our work by abstraction layer, verification method, and carry-chain coverage. Classical synthesize-then-verify systems include Denali [3], Bansal–Aiken [4], STOKE [5], Stratified Synthesis [6], and egg [7]; SMT-based peephole checking is exemplified by Alive [1], Alive2 [8], Souper [9], and PEEK [10];

dataflow filtering [11] accelerates search; recent LLM-driven peephole synthesis on LLVM IR [12] is a generative complement to deductive enumeration. We follow the verify-after-enumeration paradigm but move the checked IR from LLVM IR to dMIR, addressing the multi-return carry and overflow encoding limit that motivates this paper.

LeanMLIR [13] formalizes IR-parametric peephole verification in Lean 4, whereas this paper reports an instantiated EVM JIT rule set. Hydra [14] generalizes bit-width-independent rules but, like Alive2, builds on LLVM IR and inherits the multi-return tuple limit on the four carry-chain examples in Table 1; closing that gap via an auxiliary lifted encoding is orthogonal to our IR-extension contribution. Schett and Nagele [15] enumerate SMT-checked EVM *bytecode* rules; we instead target the JIT code-generation path with dMIR matching and x86 cleanup.

2.2 Verification Work on the Consensus-Critical Path

Albert et al. [16] give a Coq proof of AOT EVM bytecode-block rules. Our scope sits at lower abstraction layers: JIT IR and machine-code, SMT admission at synthesis time, and multi-word carry chains. Seven rules overlap: 7/83 in our set and 7/14 in their *forves* set. This supports a complementary-scope reading rather than a superset claim: the remaining 76 cover dMIR algebraic and Boolean simplifications (including ADC/SBB carry-chain rewrites that motivate the first-class carry-chain operators) plus 13 x86 cleanup rewrites, none of which are reachable from the bytecode-block abstraction.

Runtime divergence detection—Hydra Framework [17], multi-client detectors, EVM fuzzing, consensus test networks—catches faults after deployment; our workflow validates rules before deployment. Bytecode-side EVM optimization [18,19,20] is orthogonal to our JIT IR/code-generation focus.

3 SMT-Checked dMIR Peephole Rewrites

The native word length of the EVM is 256 bits, whereas general-purpose registers on current commodity hardware are 64 bits wide. An EVM-level U256 addition expands, after JIT code generation, into four machine-level additions: one ordinary 64-bit `add` followed by three carry-propagating `adc` instructions that form a *serial carry chain*. Subtraction is analogous, using one `sub` and three `sbb` instructions. Figure 1 shows the 4-limb code at the dMIR and x86-64 levels for a typical U256 addition, together with the serial carry chain that connects the four instructions.

Among all instructions emitted by the JIT, instructions directly tied to U256 multi-word arithmetic account for only **2.15%** (weighted mean across the U256-heavy subset of our 27 benchmarks—the U256-heavy cluster in Figure 2). The chain is nonetheless worth optimizing: every `adc/sbb` on the chain depends on the carry output of the previous instruction, forming a strict *serial data dependency* that defeats instruction-level parallelism.

dMIR	1:1 lowering	x86-64
<code>r0 = add(a0, b0)</code>		<code>add rax, rcx</code>
<code>r1 = ADC(a1, b1, c0)</code>	$\downarrow c_0$	<code>adc rbx, rdx</code>
<code>r2 = ADC(a2, b2, c1)</code>	$\downarrow c_1$	<code>adc rsi, rdi</code>
<code>r3 = ADC(a3, b3, c2)</code>	$\downarrow c_2$	<code>adc r8, r9</code>

Fig. 1. U256 4-limb addition in dMIR (left) and x86-64 (right). Each **ADC** lowers 1:1 to **adc**; red arrows trace the serial carry chain c_0, c_1, c_2 that the lowering preserves.

Table 1. Four typical carry-chain rules that Alive2 (based on LLVM IR) cannot directly verify for equivalence because of the multi-return tuple encoding. The first three are single **ADC/SBB** rules where the right-hand side drops the carry output; the fourth is a 4-limb carry-chain merge rule.

Rule	Rewrite
<code>adc_zero_carry</code>	$\text{ADC}(x, y, 0) \rightarrow \text{add}(x, y)$
<code>sbb_zero_borrow</code>	$\text{SBB}(x, y, 0) \rightarrow \text{sub}(x, y)$
<code>sbb_self_zero</code>	$\text{SBB}(x, x, 0) \rightarrow 0$
4-limb ADC chain $4 \times \text{uadd.w.overflow} \rightarrow \text{add.i256}$	

On U256-heavy workloads, rule hit rates reach 25–99.9% with a very uneven distribution; on non-U256-heavy benchmarks (sha1 family, `jump_around`) the U256 rule hit rate is well below that range, and gains come from x86-layer mechanical cleanup.

Hardware cost is only one side of the picture. The other side is whether existing peephole-verification frameworks can reason about this carry chain at all, which the next paragraph examines.

In LLVM IR, basic arithmetic (`add`, `sub`, `mul`) is defined only on N -bit integers `iN`; to obtain the carry bit, the programmer must call `@llvm.uadd.with.overflow.iN`, which returns a $\{\text{iN}, \text{i1}\}$ multi-return tuple. The carry is *not a native type* but hidden in the second field of a struct. Following the Alive/Alive2 [1,8] peephole verification framework, Table 1 lists four carry-chain rules that Alive2 cannot directly verify (type mismatch on the multi-return tuple, or timeout on the lifted encoding); the limitation stems from IR carry modeling, independent of solver capacity.

The four rules’ Alive2 command lines, failure messages, and tool versions (Alive2 v21.0, Z3 4.8.12) are archived in the artifact.

The bottleneck is not the SMT solver [2]: the six U256 rules without multi-return carry pass Alive2 in ~ 60 ms each.

In LLVM IR, expressing one carry-propagating addition takes five lines around `uadd.with.overflow`; in dMIR, a single **ADC** suffices. Native carry-bit support brings all four rules into the scope of automatic equivalence checking.

We first define dMIR’s bit-vector semantics, then describe enumeration and SMT admission, and finally summarize the two-tier rule organization. The JIT runs a compiled two-tier implementation: dMIR for algebraic rewrites and x86 CgIR (code-generation IR) for carry-chain cleanup.

3.1 dMIR Bit-Vector Semantics and Carry-Chain Operators

The core data type of dMIR is a parametric-width bit-vector BV_n with BV_1 carrying the carry in/out (c_{in}, c_{out}) . The carry-propagating addition has signature $ADC : BV_n \times BV_n \times BV_1 \rightarrow BV_n \times BV_1$; **SBB** has analogous borrow-propagation semantics. In Z3 [2], carry propagation zero-extends both 256-bit operands and the 1-bit carry to 257 bits, sums once, and slices: the low 256 bits are the result and bit 256 is carry out. This embeds carry propagation entirely in bit-vector arithmetic with no multi-return tuple or auxiliary predicate. The model covers dMIR arithmetic and flags; gas, memory aliasing, storage, and x86 equivalence remain outside it. The same semantics covers ordinary operators (**add**, **sub**, **mul**, **and**, **or**, **xor**, shifts); the carry-chain operator is the operator that general-purpose IRs lack.

Trusted base. The SMT check assumes the dMIR bit-vector model agrees with the JIT runtime; the 5,884 differential tests show no divergence; random-bytecode fuzzing remains future work.

3.2 Candidate Enumeration and SMT Admission

Admission check. For a candidate $LHS \rightarrow RHS$, we submit to Z3 the proposition $\exists v. LHS(v) \neq RHS(v)$. *Unsat* grants admission under fixed-width modular bit-vector semantics; *sat* or timeout rejects (sat returns a counterexample). Equivalence assumes nothing beyond modular arithmetic—no signed interpretation and no undefined-overflow assumptions. All 70 production dMIR rules pass the offline Z3 admission check, and CI re-runs this check on every change to the rule file.

Two enumeration runs. We report two separate enumeration experiments with disjoint goals. The *capacity run* fixes a single grammar—a depth-2 algebraic enumerator plus a small set of hand-written carry templates—and measures the Z3 admission rate; this characterizes solver throughput and the typical false-positive boundary. The *coverage run* extends the same algebraic enumerator with additional left-hand-side operator tops and a constant-pool widening, plus a pattern-template instantiation channel reusing the template miner configuration; this measures whether the tuned offline search space can reach the 70 hand-designed dMIR production rules, not blind recall on an unseen rule set.

The capacity run uses two complementary paths: (i) *depth-2 algebraic enumeration*—all left/right combinations of depth ≤ 2 over the dMIR arithmetic and Boolean operators, covering common algebraic identities; (ii) *carry-template enumeration*—multi-word carry-chain candidates constructed by hand around the carry-chain operators.

Throughput and admission rate. The capacity run admits 765/775 algebraic candidates (98.7%, 10 timeouts) and 17/78 carry templates (21.8%, 61 semantic rejections): carry-template admission is limited by 61 semantic rejections, mostly width-mismatch edge cases (full breakdown in the artifact), while the algebraic side has only 10 timeouts and zero semantic rejections. Full data: Table 3. These candidates measure enumeration capacity; only the curated production rules are loaded into the JIT at runtime.

Production rule coverage (methodology). The coverage run asks a different question: does the enumeration search space contain the 70 hand-designed production dMIR rules? We compare canonicalized left-hand sides. The *alpha-normalized* canonical form renames variables to a fixed sequence $x_1, x_2, \dots$ in traversal order, so that two rules differing only in operand naming map to the same representative; commutative pairs (e.g. $\text{or}(x, y)$ and $\text{or}(y, x)$) likewise collapse to one representative. A production rule counts as *covered* when at least one enumerated rule has the same canonical left-hand side; the *false-miss rate* is the fraction of production rules with no such match. The match is on canonical left-hand sides only—the enumerated right-hand side is verified equivalent to the left-hand side by Z3 but is not required to be syntactically identical to the production rule’s right-hand side.

Counterexamples. Two rejected carry templates illustrate why SMT admission is essential on the carry chain. $\text{SBB}(x, x, c_f) \rightarrow 0$ fails at $x=0, c_f=1$ (LHS is $0\text{xFF} \dots \text{F}$; correct form $\text{sub}(0, c_f)$); $\text{SBB}(x, 0, c_f) \rightarrow \text{sub}(0, c_f)$ fails at $x=1, c_f=0$ (correct form $\text{sub}(x, c_f)$). Both reflect modular-carry traps: ignored carry input and confused operand polarity. The artifact contains ~ 2400 lines of Python (enumerator, template miner, canonical comparator, Z3 admission checks), the rule-set JSON, coverage output, and unit tests for canonicalization/dedup.

Two-tier rule set. The dMIR layer handles machine-independent algebraic simplification (all 70 dMIR rules pass Z3); the x86 CgIR layer handles hardware-specific cleanup on registers, flags, and branches (13 syntactic rewrites with **no separate SMT script**, covered by the matched 5,884-test suites and Table 4); a formal x86 SMT model remains future work.

Table 2 groups the rules by logical class. Boolean normalization and algebraic identities together cover most dMIR rules—these are precisely the patterns the canonical-form enumerator is designed to cover. Identity-elimination and shift-zero patterns admit Z3-discharged proofs in milliseconds, while the carry-chain class is the one that motivates the operator extension introduced earlier in this section.

Selection asymmetry. The 70 dMIR production rules are selected manually from recurring profile hotspots and lowering patterns, then submitted one by one to Z3; this is hotspot-guided curation, not a claim of systematic optimality. The enumeration pipeline is a measurement channel: it does not drive production rule maintenance, and the 1966 admitted candidates are not automatically promoted

Table 2. Classification of the two-tier peephole rewrite rules by evidence type. The dMIR rules are checked for equivalence by Z3 on the bit-vector semantic model; the x86 CgIR rules are covered by matched differential tests, not by SMT.

Layer	Evidence	Category	Count
dMIR	Z3 equivalence check	Boolean normalization	42
		Algebraic identity	15
		Identity elimination	10
		Constant folding	2
		Dead-code elimination	1
		Subtotal	70
x86 CgIR	Differential tests	Redundant compare/test cleanup	8
		Branch-fallthrough cleanup	2
		No-op elimination	2
		test-setcc merge	1
		Subtotal	13
Total			83

Table 3. Output of the enumeration/admission pipeline: candidate counts, Z3 passes, SAT counterexamples, timeouts, and admission rate. The two enumeration paths are described in detail in §3.2.

Candidate source	Input	Pass	SAT cex	Timeout	Admission rate
Algebraic enumeration	775	765	0	10	98.7%
Carry templates	78	17	61	0	21.8%

SAT counterexamples are semantic rejections; timeouts are conservatively rejected without counterexamples.

to production. The x86-layer rules are covered by code review and matched test suites.

4 Evaluation

4.1 Enumeration and Admission Output vs. the General-Purpose IR

Table 3 reports the enumeration/admission output of the capacity run; Alive2’s verification on the same rule sample is summarized below.

Z3 discharges the hardest 256-bit queries in ~ 60 ms and rechecks all 70 dMIR rules in ~ 0.5 s, so width is not the bottleneck. On a 10-rule U256 capacity-run sample, Alive2 clears the 6 without multi-return carry but reports type mismatch or timeout on the 4 `adc/sbb` cases. This sample is separate from the four production-style cases in Table 1; both expose the same tuple cause.

Production rule coverage. Using the canonical-LHS methodology, the coverage run produces **1966** Z3-admitted rewrite rules: **1856** from the extended algebraic enumerator and **110** from pattern-template instantiation. The pattern-template path starts from 2964 candidates, of which 110 survive Z3 verification and deduplication against the enumerator output and the existing rule file. On the 70 hand-designed production rules, the run matches **69/69** unique canonical patterns—one commutative-twin pair, $\text{or}(x, y)$ versus $\text{or}(y, x)$, shares a single canonical representative that both production names match—equivalently **70/70** production rule names; the false-miss rate against the production set is therefore **0/70**.

Scope of the coverage claim. The claim has two boundaries. First, the result measures coverage of the *existing* production rule set; the extended enumerator’s left-hand-side operator tops and constant pool were broadened in light of an earlier 0/70 baseline that exposed mechanical gaps (missing tops such as `not/select/shl/ushr/sshr` as left-hand-side roots, missing small constants, and structural-equality dedup of forms such as `and(x, x)`). Reaching 70/70 therefore demonstrates that the production rules lie inside an enumerable and Z3-discharged search space; it does not demonstrate blind recall of an unseen target. Second, coverage is defined on canonical left-hand sides; the enumerated right-hand side is verified equivalent to its own left-hand side by Z3 but is not required to be syntactically identical to the production rule’s right-hand side. Together with the admission rate above, this result shows the tuned search space is broad enough to contain every hand-designed rule, while 1966 of its members pass Z3 verification; it does not claim that the non-production members are useful runtime rewrites.

4.2 End-to-End Performance

Setup. Intel Core Ultra 7 265K/Ubuntu 22.04; evmone-bench, baseline (upstream `main`) vs. rules-enabled side, 20 runs (30 if $\text{CV} \geq 3\%$), bootstrap CIs ($B=10\,000$), pinned CPU, turbo off.

Across all 27 benchmarks, the full 83-rule set improves the geometric mean by **+7.24%** (95% CI [+2.59%, +12.11%]); the positive-speedup cases are shown in Figure 2. Peak speedup is **+25.79%** on `JUMPDEST_n0/empty`. The `sha1` family gains +19.59%–+24.81%, and `blake2b_huff` gains +10.41%–+11.54%.

Hit rate and speedup are decoupled: `mg_mload` matches nearly every U256 instruction yet gains under 8%, while `sha1` hits only 7.5% of opcodes yet gains over 20% from x86 branch and compare-merge cleanup. The two layers complement each other: multi-word rules concentrate gains on U256-heavy contracts, x86 cleanup yields broader gains on ordinary code.

Compilation overhead. Across 776 JIT unit-test modules, median per-module compile time is 0.45 ms (p95 0.87 ms) covering the full loading–analysis–codegen pipeline; this is an upper bound on the whole pipeline rather than the incremental rewrite-pass cost (per-pass attribution remains future work).

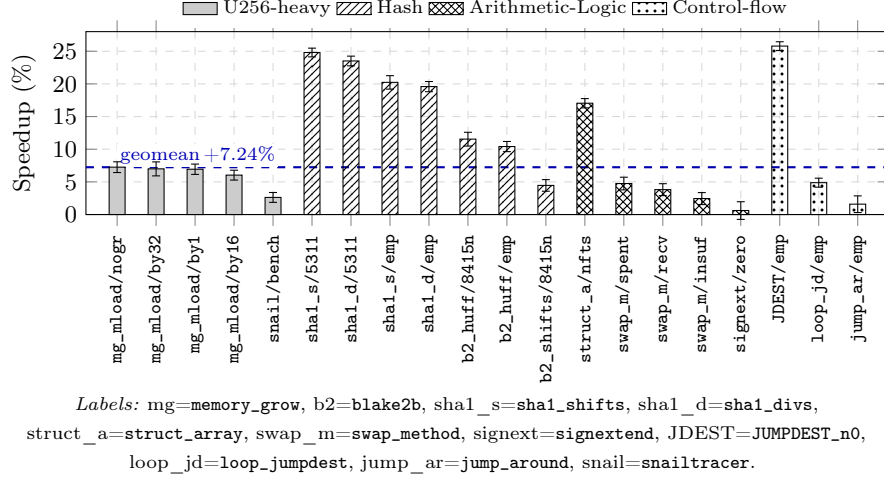


Fig. 2. Positive-speedup benchmark configurations from enabling the peephole rewrite rule set over the upstream-main baseline (error bars are 95% bootstrap confidence intervals, $B=10\,000$). The horizontal line at zero marks the no-change baseline; the dashed line at +7.24% marks the 27-benchmark geometric-mean speedup. From left to right, the benchmarks are grouped into four workload clusters: U256-heavy, Hash, Arithmetic-Logic, and Control-flow.

4.3 Correctness and Rule Coverage

Both sides agree item-by-item across the four differential categories of Table 4 (5884/5884 total). Independently, `evmone-statetest` on the Cancun fork reaches the same matrix under multipass JIT and interpreter modes.¹ The suite covers Cancun state tests and EVM unit tests, but we do not claim opcode- or gas-branch coverage completeness.

CI re-runs the same admission check on every change to the rule file; the timing is distinct from the per-module JIT compile time reported above.

¹ Cancun is selected via `evmone-statetest -k fork_Cancun`; Prague tests are excluded because DTVM does not yet support that fork.

Table 4. Results on four differential test categories for the rules-enabled side (83 added rules) and the baseline side (upstream main): passes/total. Matching numbers in both columns indicate that the enabled rule set introduces no correctness regression.

Test dimension	Enabled	Baseline	Suite source
Unit (JIT)	223/223	223/223	EVM spec subset
Unit (interpreter)	215/215	215/215	Run list
statetest (JIT)	2723/2723	2723/2723	Cancun 101 suites
statetest (interpreter)	2723/2723	2723/2723	Cancun 101 suites
Total	5884/5884	5884/5884	—

5 Conclusion

This paper extends a domain-specific JIT IR (dMIR) with first-class carry and overflow operators and builds an SMT-checked peephole layer on top of it: 70 dMIR rules pass Z3 admission over arithmetic and flags, and 13 x86 cleanup rules are covered by matched differential testing. The combination delivers a +7.24% geometric-mean speedup on evmone-bench while preserving byte-identical final state across 5,884 Cancun differential tests, with correctness bounded by suite coverage. Future work includes a formal x86 model to replace the differential-test fallback at the cleanup layer and cross-engine validation against evmone and revm.

Data and Code Availability

Rule sets, offline Z3 admission checks, evaluation scripts, and data are available in an anonymous repository upon request to the program chairs and will be de-anonymized upon acceptance. CI runs the production dMIR validation tests on every change to the rule file.

References

1. Nuno P. Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. Provably correct peephole optimizations with alive. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2015.
2. Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008.
3. Rajeev Joshi, Greg Nelson, and Keith Randall. Denali: A goal-directed superoptimizer. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI)*, 2002.
4. Sorav Bansal and Alex Aiken. Automatic generation of peephole superoptimizers. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2006.
5. Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013.

6. Stefan Heule, Eric Schkufza, Rahul Sharma, and Alex Aiken. Stratified synthesis: Automatically learning the x86-64 instruction set. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2016.
7. Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.*, 5(POPL), 2021.
8. Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021.
9. Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Jubi Taneja, and John Regehr. Souper: A synthesizing superoptimizer, 2017.
10. Eric Mullen, Daryl Zuniga, Zachary Tatlock, and Dan Grossman. Verified peephole optimizations for CompCert. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2016.
11. Manasij Mukherjee, Pranav Kant, Zhengyang Liu, and John Regehr. Dataflow-based pruning for speeding up superoptimization. *Proc. ACM Program. Lang.*, 4(OOPSLA), 2020.
12. Zhenyang Xu, Hongxu Xu, Yongqiang Tian, Xintong Zhou, and Chengnian Sun. LPO: Discovering missed peephole optimizations with large language models. In *Proceedings of the 31st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026.
13. Siddharth Bhat, Alex Keizer, Chris Hughes, Andrés Goens, and Tobias Grosser. Verifying peephole rewriting in SSA compiler IRs. In *15th International Conference on Interactive Theorem Proving (ITP)*, volume 309 of *LIPIcs*, pages 9:1–9:20, 2024.
14. Manasij Mukherjee and John Regehr. Hydra: Generalizing peephole optimizations with program synthesis. *Proc. ACM Program. Lang.*, 8(OOPSLA1), 2024.
15. Maria A. Schett and Julian Nagele. Populating the peephole optimizer of a smart contract compiler. In *2nd Workshop on Formal Methods for Blockchains (FMBC 2020)*, OASICS, 2020.
16. Elvira Albert, Samir Genaim, Daniel Kirchner, and Enrique Martin-Martin. Formally verified EVM block-optimizations. In *Computer Aided Verification (CAV)*, volume 13966 of *LNCS*, pages 176–189, 2023.
17. Lorenz Breidenbach, Phil Daian, Florian Tramèr, and Ari Juels. Hydra: Principled bug bounties for smart contract security. In *27th USENIX Security Symposium (USENIX Security 18)*, 2018.
18. Elvira Albert, Pablo Gordillo, Albert Rubio, and Maria A. Schett. Synthesis of super-optimized smart contracts using Max-SMT. In *Computer Aided Verification (CAV)*, LNCS, 2020.
19. Xiaowen Hu, David Zhao, and Bernhard Scholz. Synthesizing efficient super-instruction sets for Ethereum virtual machine. In *Proceedings of the 16th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL)*, 2024.
20. Matheus Araújo Aguiar, Elvira Albert, Samir Genaim, Pablo Gordillo, Alejandro Hernández-Cerezo, Daniel Kirchner, and Albert Rubio. Neural-guided superoptimization in Ethereum. *Information and Software Technology*, 2025.